

Guest Lecture at USN, Norway • Applied 2-hour session (15th Sept 2026)

Build an Image Classifier with Transfer Learning

with Keras: from pixels to predictions

Ayan Chatterjee, Ph.D.
Scientist, NILU Research Lab, Kjeller, Norway

Case study: waste image classification with Kaggle datasets

Background • CNNs • Transfer learning • Hands-on



2-hour applied activity plan

0–10	Motivation	Show the classification problem: pixels → class probabilities
10–30	Background	Deep learning, training loop, optimization, and model families
30–45	Vision concepts	Pixels as tensors, CNNs, layers, transfer learning
45–65	Data prep	Load image folders, inspect class counts, augment training data
65–90	Build + train	Keras MobileNetV2 classifier and train frozen-head model
90–105	Evaluate	Confusion matrix, inspect failure cases, discuss uncertainty
105–120	Improve	Fine-tuning idea, deployment checklist, student reflection

Session map: from deep learning to classification

1

Background

Why deep learning; how neural networks operate; optimization intuition

2

Vision concepts

Pixels as tensors; image classification; CNN layers and features

3

Keras workflow

Load data; augment; build a classifier; train and fine-tune

4

Evaluation

Confusion matrix; error analysis; what to improve before deployment

Learning target: students can explain and run a complete image-classification pipeline.

Why deep learning is needed

Real-world data is messy

Images vary by lighting, angle, scale, background, occlusion, camera type, and noise.

Rules do not scale

Hand-coding “if edge + color + texture” quickly becomes brittle for natural images.

Features are learned

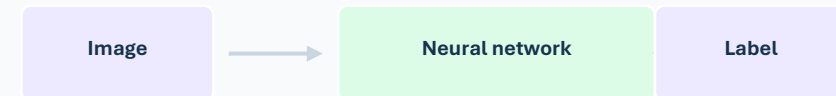
Deep learning learns useful internal representations directly from examples.

Traditional ML often depends on handcrafted features; deep learning learns multiple levels of representation from data.

Handcrafted pipeline



Deep learning pipeline



Evolution of computer vision

- Rules** if/then logic
- Features** SIFT, HOG, color
- CNNs** learned filters
- Transfer** reuse pretrained features

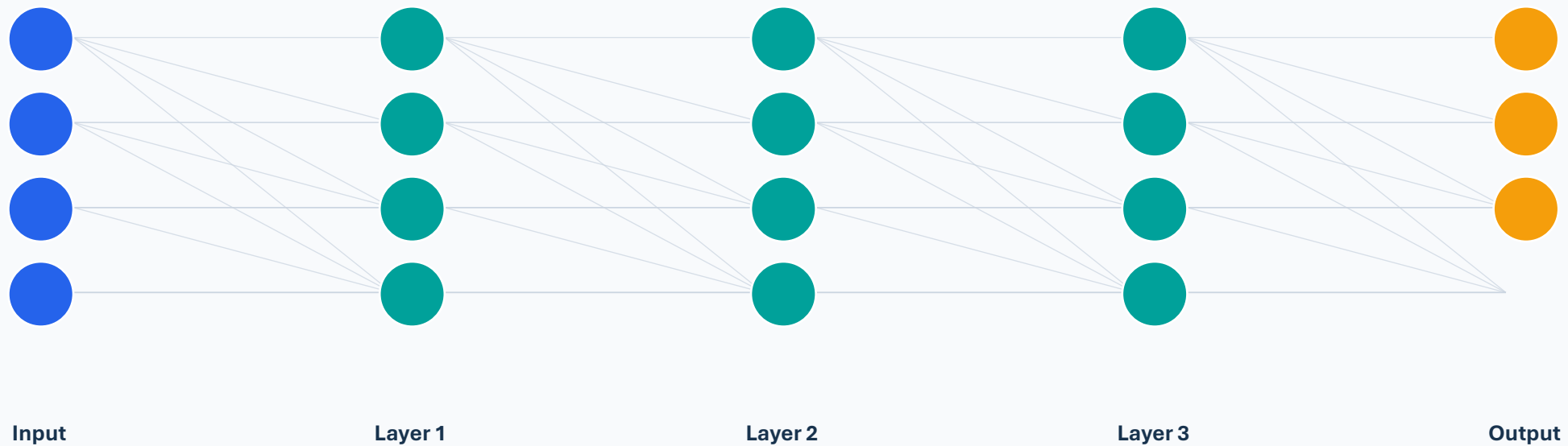
IF cat(x) AND furry(x)
THEN animal(x)

IF bird(x) AND flies(x)
THEN animal(x)

...

How a neural network operates

A network is a sequence of simple mathematical transformations. Training finds weights that make predictions closer to the labels.



Weights

Numbers inside the model that control each transformation.

Activation

Non-linear function, e.g. ReLU, that lets networks model complex patterns.

Prediction

Class probabilities, usually produced with softmax in multiclass tasks.

The training loop: predict, compare, update



Forward pass

Input flows through the model to produce predictions.

Loss

A loss function measures how wrong the predictions are.

Backpropagation

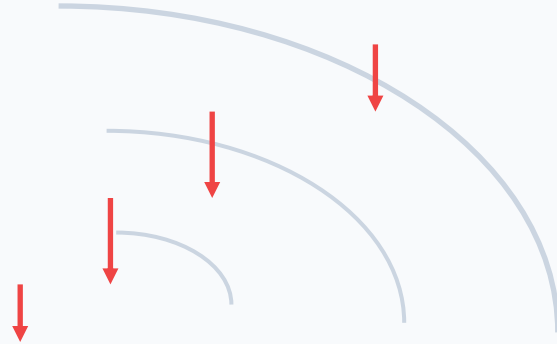
Gradients estimate how each weight contributed to the error.

Optimizer step

Weights are updated to reduce future loss.

Repeat over many mini-batches until validation performance stops improving.

How optimization works: downhill with noisy steps



Loss surface

Learning rate

Step size. Too high can overshoot; too low can be slow.

Mini-batches

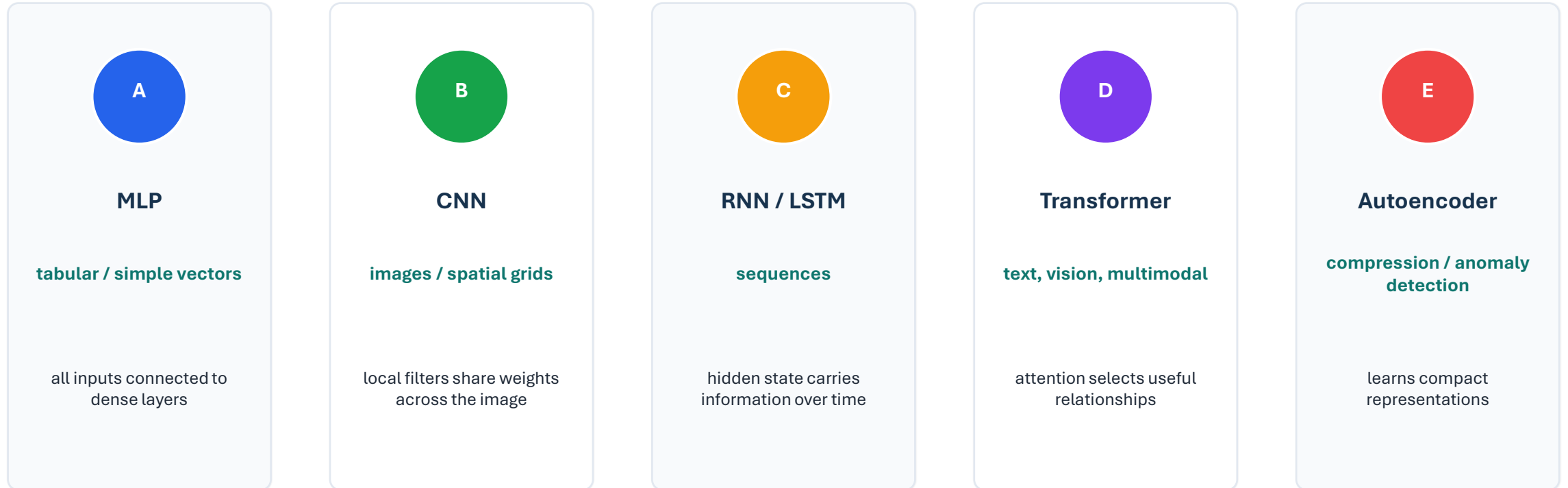
Each batch gives a noisy but useful estimate of the direction.

Adam optimizer

Adapts updates using moving averages of gradients and squared gradients.

Key intuition: optimization is not memorization. It is repeated adjustment of millions of parameters to reduce loss.

Different deep learning model families



For today: focus on CNNs and pretrained CNN backbones for image classification.

Keras mental model: layers → model → compile → fit

```
from tensorflow import keras
from tensorflow.keras import layers

model = keras.Sequential([
    layers.Input(shape=(224, 224, 3)),
    layers.Rescaling(1./255),
    layers.Conv2D(32, 3, activation="relu"),
    layers.MaxPooling2D(),
    layers.Flatten(),
    layers.Dense(num_classes, activation="softmax")
])

model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.fit(train_ds, validation_data=val_ds, epochs=5)
```

Layer

A transformation such as convolution, pooling, dropout, or dense.

Model

A graph of layers with inputs and outputs.

Compile

Choose loss, optimizer, and metrics.

Fit

Train the model on batches of data.

Why computer vision is different from ordinary tabular data

Spatial structure

Nearby pixels are related: edges, corners, textures, object parts.

Huge inputs

$224 \times 224 \times 3$ image = 150,528 numbers before the model even starts.

Shared patterns

A bottle edge is useful whether it appears left, right, high, or low.

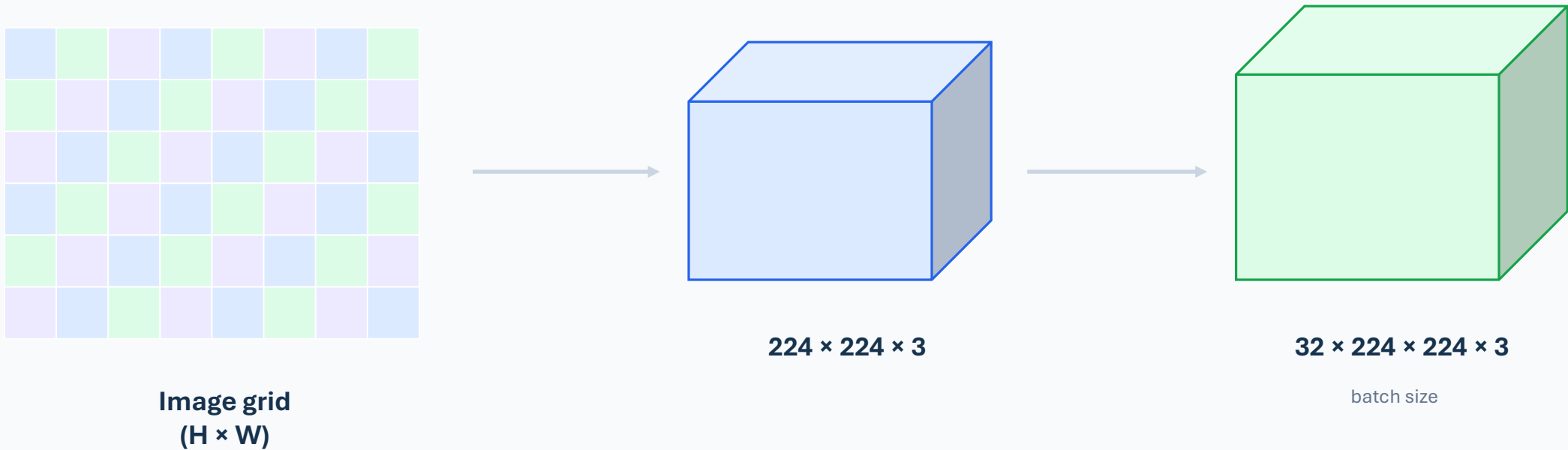
Robustness needed

Models must generalize to changes in light, angle, background, and scale.

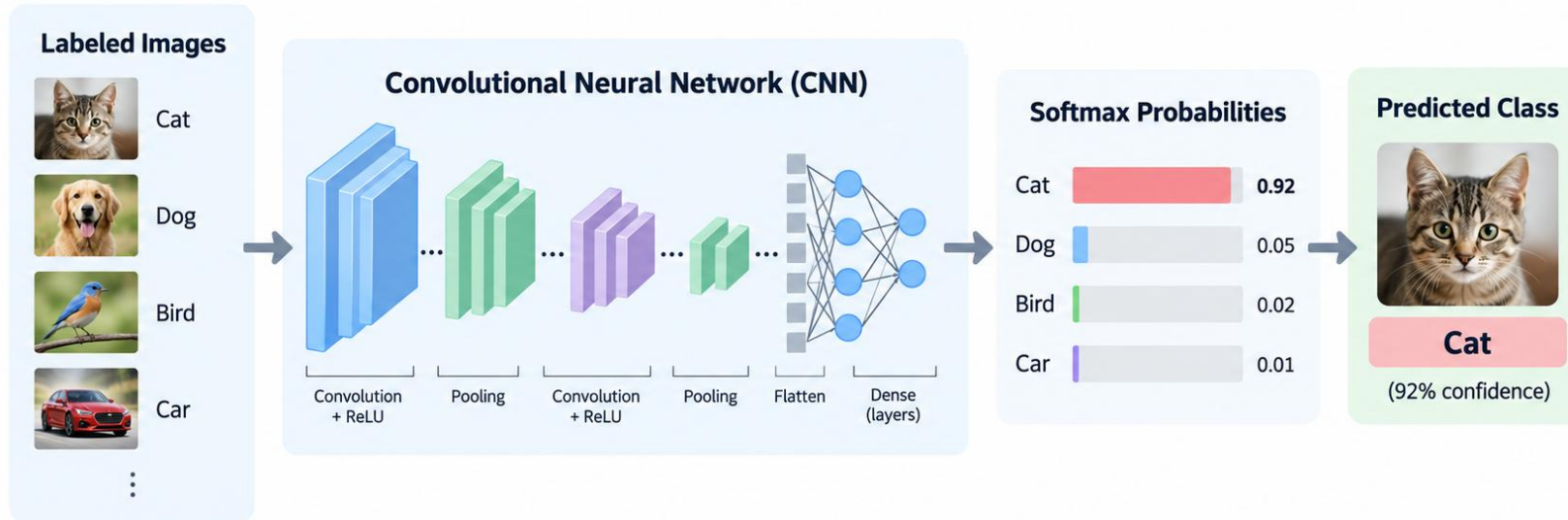
CNNs encode useful assumptions: locality + weight sharing + hierarchical features

From pixels to tensors

A color image becomes a 3D array: height × width × channels. A batch adds one more dimension.



Pixels to classifications: the complete pipeline



Key idea: the model transforms raw pixels into feature maps, then into class probabilities.

Data preparation: the make-or-break step

```
train_ds = keras.utils.image_dataset_from_directory(  
    "dataset/train",  
    image_size=(224, 224),  
    batch_size=32,  
    seed=42  
)  
  
val_ds = keras.utils.image_dataset_from_directory(  
    "dataset/validation",  
    image_size=(224, 224),  
    batch_size=32,  
    seed=42  
)
```

Folder labels

Each class is a subfolder; folder names become labels.

Resize

Batches need a consistent image size.

Split

Keep validation and test data separate from training.

Cache/prefetch

Speed up input loading so training is not waiting on disk.

Never evaluate on images used for training.

Data augmentation: teach the model useful invariances

```
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.08),
    layers.RandomZoom(0.15),
])

# Apply augmentation only during training
x = data_augmentation(inputs)
```

Flip

object may appear mirrored

Rotation

camera or object may tilt

Zoom

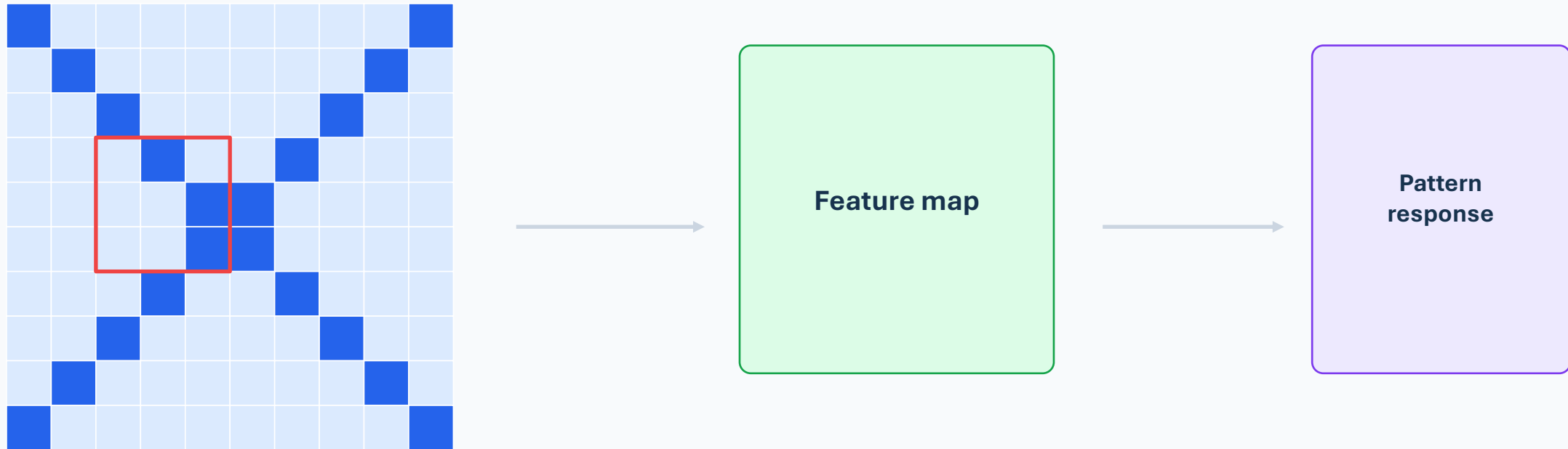
object distance varies

Not every transform is valid

e.g. color inversion may be unrealistic for sorting waste

Augmentation increases training diversity without collecting new images.

CNN intuition: a small filter scans the image



Locality

A filter sees a small patch at a time.

Weight sharing

The same filter is reused across the image.

Feature maps

Each filter produces a map showing where a pattern appears.

Different layers in a CNN classifier

1

Conv2D

learns filters for edges, textures,
and object parts

2

ReLU

keeps positive activations and adds
non-linearity

3

Pooling

shrinks feature maps while keeping
strong responses

4

BatchNorm

stabilizes activation statistics
during training

5

Dropout

randomly disables activations to
reduce overfitting

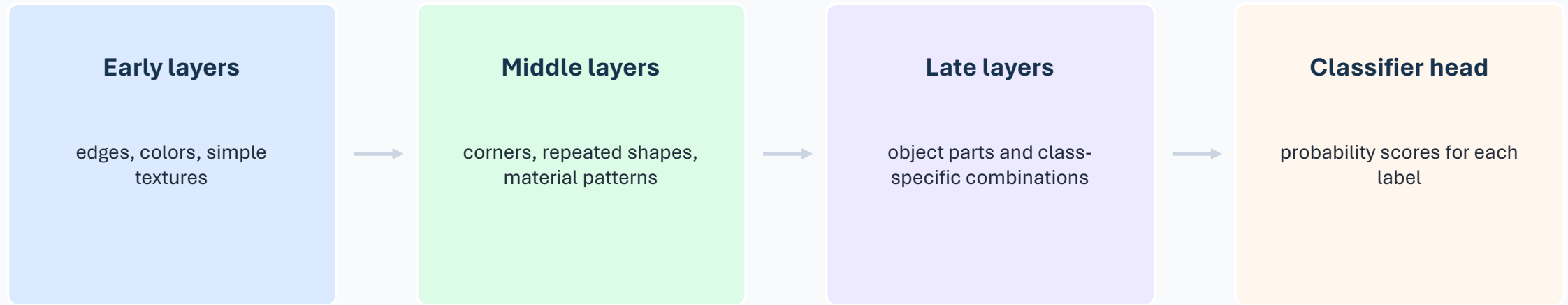
6

Dense + Softmax

turns features into class
probabilities

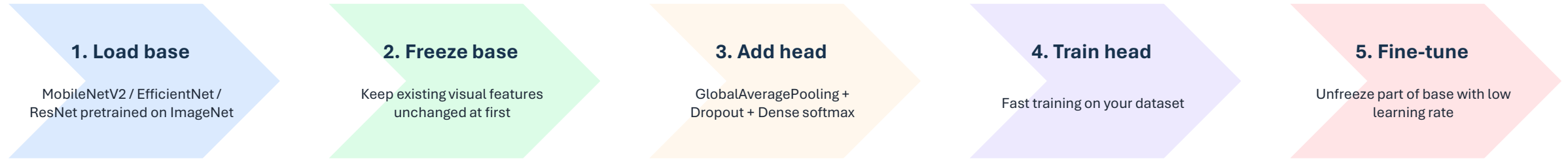
Not every model uses every layer, but these names appear constantly in Keras examples.

What CNN layers learn: a hierarchy of features



This is why transfer learning works: early and middle visual features are reusable across many image tasks.

Transfer learning: practical recipe



Why it matters

Small student datasets are rarely enough to train strong CNN features from scratch.

Best for teaching

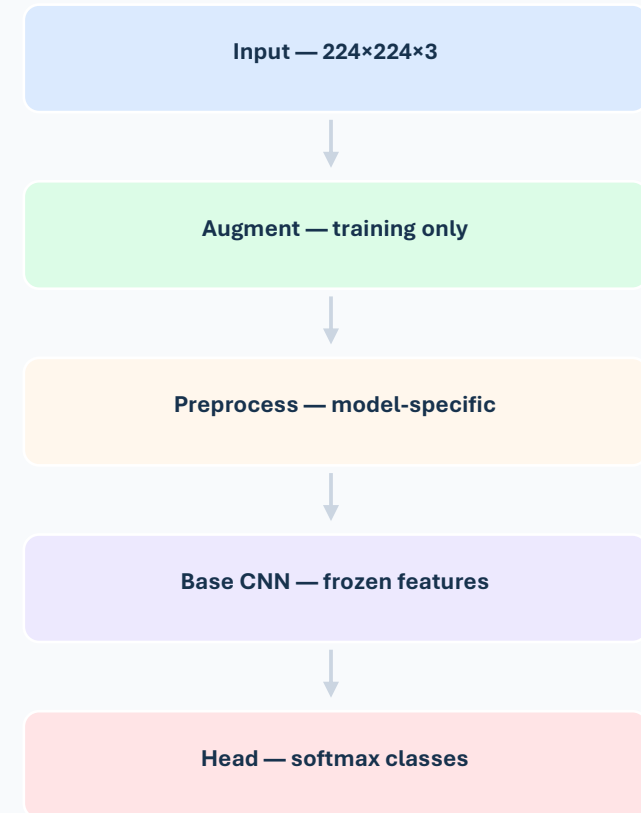
Students get useful results quickly and can focus on data, evaluation, and error analysis.

Caution

Fine-tuning too aggressively can destroy useful pretrained features.

Keras transfer-learning model

```
base = keras.applications.MobileNetV2(  
    input_shape=(224, 224, 3),  
    include_top=False,  
    weights="imagenet"  
)  
base.trainable = False  
  
inputs = keras.Input(shape=(224, 224, 3))  
x = data_augmentation(inputs)  
x = keras.applications.mobilenet_v2.preprocess_input(x)  
x = base(x, training=False)  
x = layers.GlobalAveragePooling2D()(x)  
x = layers.Dropout(0.2)(x)  
outputs = layers.Dense(num_classes, activation="softmax")(x)  
model = keras.Model(inputs, outputs)
```



Training and fine-tuning workflow

```
model.compile(  
    optimizer=keras.optimizers.Adam(1e-3),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)  
history = model.fit(train_ds, validation_data=val_ds, epochs=5)  
  
# Optional: fine-tune slowly  
base.trainable = True  
model.compile(  
    optimizer=keras.optimizers.Adam(1e-5),  
    loss="sparse_categorical_crossentropy",  
    metrics=["accuracy"]  
)  
model.fit(train_ds, validation_data=val_ds, epochs=3)
```

Phase 1: frozen base

Train only the new classifier head.

Phase 2: fine-tune

Unfreeze part or all of the base with a much lower learning rate.

Early stopping

Stop if validation performance no longer improves.

Watch overfitting

Training improves but validation gets worse.

Evaluate beyond accuracy

Confusion matrix example

		Predicted →		
		plastic	paper	glass
Actual	plastic	21	3	1
	paper	2	22	1
	glass	4	2	18

Accuracy

Overall proportion correct; can hide rare-class problems.

Inspect errors

Show misclassified images and ask why.

Precision

When the model says “plastic,” how often is it right?

Class imbalance

A majority class can dominate accuracy.

Recall

Of all real plastic items, how many did we detect?

Confidence

Top probability is not always trustworthy.

Debugging model mistakes

Data issue?

Wrong label, blurry image, duplicated samples, class imbalance.

Preprocessing issue?

Wrong input size, color channel, or application-specific scaling.

Model issue?

Too few epochs, too much fine-tuning, underfitting, overfitting.

Task issue?

Labels are ambiguous: paper cup, plastic-coated paper, mixed materials.

For students: every wrong prediction is evidence. Use it to improve data, labels, model, or deployment rules.

From classroom model to real-world classifier

Data shift

Will new cameras, lighting, and backgrounds differ from training?

Confidence threshold

What should happen when the model is unsure?

Cost of errors

Which confusions matter most in the application?

Human review

Which predictions should be routed to a person?

Monitoring

How do we detect model drift after deployment?

Ethics / privacy

Could images contain people or sensitive context?

Key takeaways

From deep learning foundations to Keras image classification

Deep learning: learns useful representations from examples.

CNNs: use filters to exploit spatial structure in images.

Transfer learning: makes strong models possible with small datasets.

Data preparation: often matters more than model choice.

Evaluation: requires error analysis, not just accuracy.

Deployment: needs uncertainty, monitoring, and human fallback.

Challenge: one improvement to the data, one to the model, one to the evaluation.